Java Programming

Core Concepts & Solutions

Factorial • Palindrome • Fibonacci • Prime • Reverse Array • Sorting

F! 1. Factorial

Concept

The factorial of a non-negative integer n (written n!) is the product of all positive integers from 1 to n. Example: 5! = 5 x 4 x 3 x 2 x 1 = 120. Both iterative and recursive solutions are shown.

Java Code

```
public class Factorial {

    // Iterative approach

    static long factIterative(int n) {

        long result = 1;

        for (int i = 2; i <= n; i++) result *= i;

        return result;

    }


    // Recursive approach

    static long factRecursive(int n) {

        return (n <= 1) ? 1 : n * factRecursive(n - 1);

    }


    public static void main(String[] args) {

        int n = 6;

        System.out.println(n + "! (iterative) = " + factIterative(n));

        System.out.println(n + "! (recursive) = " + factRecursive(n));

    }

}
```

Output

```
6! (iterative) = 720

6! (recursive) = 720
```

PL 2. Palindrome

Concept

A palindrome reads the same forwards and backwards. For strings: compare with its reverse. For integers: reverse the digits and compare with the original value.

Java Code

```
public class Palindrome {

    static boolean isStringPalindrome(String s) {

        String rev = new StringBuilder(s).reverse().toString();

        return s.equalsIgnoreCase(rev);

    }


    static boolean isNumberPalindrome(int n) {

        int original = n, reversed = 0;

        while (n > 0) {

            reversed = reversed * 10 + n % 10;

            n /= 10;

        }

        return original == reversed;

    }


    public static void main(String[] args) {

        System.out.println("'racecar': " + isStringPalindrome("racecar"));

        System.out.println("'hello'  : " + isStringPalindrome("hello"));

        System.out.println("12321    : " + isNumberPalindrome(12321));

        System.out.println("12345    : " + isNumberPalindrome(12345));

    }

}
```

Output

```
'racecar': true
```

```
'hello'  : false

12321    : true

12345    : false
```

FB 3. Fibonacci Series

Concept

Each Fibonacci number is the sum of the two preceding ones: 0, 1, 1, 2, 3, 5, 8, 13 … Below are iterative, recursive, and dynamic-programming (memoization) versions.

Java Code

```
import java.util.HashMap;

public class Fibonacci {

    // Iterative

    static void printSeries(int n) {

        int a = 0, b = 1;

        System.out.print("Series: ");

        for (int i = 0; i < n; i++) {

            System.out.print(a + (i < n-1 ? ", " : ""));

            int temp = a + b; a = b; b = temp;

        }

        System.out.println();

    }


    // Recursive with memoization

    static HashMap<Integer,Long> memo = new HashMap<>();

    static long fib(int n) {

        if (n <= 1) return n;

        if (memo.containsKey(n)) return memo.get(n);

        long result = fib(n-1) + fib(n-2);

        memo.put(n, result);

        return result;

    }
```

```
    public static void main(String[] args) {
        printSeries(10);
        System.out.println("fib(10) = " + fib(10));
    }
}
```

Output

```
Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
fib(10) = 55
```

PR 4. Prime Number

Concept

A prime number is divisible only by 1 and itself. The optimised check tests divisors only up to the square root of n. The Sieve of Eratosthenes finds all primes up to a limit efficiently.

Java Code

```
import java.util.Arrays;

public class PrimeNumber {
    // Check single number
    static boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2)
            if (n % i == 0) return false;
        return true;
    }

    // Sieve of Eratosthenes
    static void sieve(int limit) {
        boolean[] isComposite = new boolean[limit + 1];
        for (int i = 2; i * i <= limit; i++)
```

```
            if (!isComposite[i])
                for (int j = i*i; j <= limit; j += i)
                    isComposite[j] = true;
        System.out.print("Primes up to " + limit + ": ");
        for (int i = 2; i <= limit; i++)
            if (!isComposite[i]) System.out.print(i + " ");
        System.out.println();
    }

    public static void main(String[] args) {
        System.out.println("isPrime(17): " + isPrime(17));
        System.out.println("isPrime(18): " + isPrime(18));
        sieve(50);
    }
}
```

Output

```
isPrime(17): true
isPrime(18): false
Primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

RA 5. Reverse Array

Concept

Reversing an array can be done in-place using the two-pointer technique (swap from both ends towards the centre) in O(n) time and O(1) space. A copy-based approach is also shown.

Java Code

```
import java.util.Arrays;

public class ReverseArray {
    // In-place two-pointer
    static void reverseInPlace(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
```

```
            int temp  = arr[left];
            arr[left] = arr[right];
            arr[right]= temp;
            left++; right--;
        }
    }

    // Return a new reversed array
    static int[] reverseCopy(int[] arr) {
        int[] result = new int[arr.length];
        for (int i = 0; i < arr.length; i++)
            result[i] = arr[arr.length - 1 - i];
        return result;
    }

    public static void main(String[] args) {
        int[] original = {1, 2, 3, 4, 5, 6};
        System.out.println("Original : " + Arrays.toString(original));
        reverseInPlace(original);
        System.out.println("Reversed : " + Arrays.toString(original));

        int[] arr2 = {10, 20, 30, 40, 50};
        System.out.println("Copy rev : " + Arrays.toString(reverseCopy(arr2)));
    }
}
```

Output

```
Original : [1, 2, 3, 4, 5, 6]
Reversed : [6, 5, 4, 3, 2, 1]
Copy rev : [50, 40, 30, 20, 10]
```

ST 6. Sorting Basics

Concept

Three fundamental sorting algorithms: Bubble Sort (O(n²), simple), Selection Sort (O(n²), minimal swaps), and Insertion Sort (O(n²) worst / O(n) best, great for nearly-sorted data). Java's built-in Arrays.sort() uses TimSort (O(n log n)).

Java Code

```
import java.util.Arrays;

public class SortingBasics {

    // 1. Bubble Sort
    static void bubbleSort(int[] a) {
        int n = a.length;
        for (int i = 0; i < n-1; i++)
            for (int j = 0; j < n-i-1; j++)
                if (a[j] > a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
    }

    // 2. Selection Sort
    static void selectionSort(int[] a) {
        for (int i = 0; i < a.length-1; i++) {
            int minIdx = i;
            for (int j = i+1; j < a.length; j++)
                if (a[j] < a[minIdx]) minIdx = j;
            int t = a[minIdx]; a[minIdx] = a[i]; a[i] = t;
        }
    }

    // 3. Insertion Sort
    static void insertionSort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int key = a[i], j = i - 1;
            while (j >= 0 && a[j] > key) { a[j+1] = a[j]; j--; }
```

```
            a[j+1] = key;
        }
    }

    public static void main(String[] args) {
        int[] arr1 = {64,34,25,12,22,11,90};
        int[] arr2 = arr1.clone();
        int[] arr3 = arr1.clone();
        bubbleSort(arr1);    System.out.println("Bubble   : "+Arrays.toString(arr1));
        selectionSort(arr2); System.out.println("Selection: "+Arrays.toString(arr2));
        insertionSort(arr3); System.out.println("Insertion: "+Arrays.toString(arr3));
        // Built-in
        int[] arr4 = {5,3,8,1,9,2};
        Arrays.sort(arr4);   System.out.println("Built-in : "+Arrays.toString(arr4));
    }
}
```

Output

```
Bubble   : [11, 12, 22, 25, 34, 64, 90]
Selection: [11, 12, 22, 25, 34, 64, 90]
Insertion: [11, 12, 22, 25, 34, 64, 90]
Built-in : [1, 2, 3, 5, 8, 9]
```

Topic	Key Technique	Complexity
Factorial	Iteration / Recursion	O(n)
Palindrome	StringBuilder.reverse()	O(n)
Fibonacci	Memoization / DP	O(n)
Prime Number	Sqrt check + Sieve	O(sqrt n) / O(n log log n)
Reverse Array	Two-pointer swap	O(n)
Sorting	Bubble / Selection / Insertion	$O(n^2)$